task_11.py - C:/Users/josth/AppData/Local/Programs/Python/Python313/task_11.py (3.13.17)

File Edit Format Run Options Window Help

```
# TASK 11: Recommend and Defend the Final Design

import json
import os
import time
from decimal import Decimal, ROUND_HALF_UP

# Create a JSON rate card file
RATE_FILE = "rate_card.json"

default_rate_card = {
    "base": "30.00",
    "fat": "0.40",
    "snf": "0.30",
    "reference_fat": "3.5",
    "reference_snf": "8.5"
}

if not os.path.exists(RATE_FILE):
    with open(RATE_FILE, "w") as file:
        json.dump(default_rate_card, file, indent=4)


# Load the rate card from the JSON file
def load_rate_card():
    start = time.perf_counter()

    with open(RATE_FILE, "r") as file:
        data = json.load(file)

    end = time.perf_counter()

    file_size = os.path.getsize(RATE_FILE)
    load_time = (end - start) * 1000000

    return data, file_size, load_time


RATE_CARD, file_size, load_time = load_rate_card()


# Calculate milk rate
def calculate_milk_rate(fat, snf):
    fat = Decimal(str(fat))
    snf = Decimal(str(snf))
```

```
    if not Decimal("3.0") <= fat <= Decimal("6.0"):
        raise ValueError("Fat must be between 3.0 and 6.0")

    if snf <= 0:
        raise ValueError("SNF must be positive")

    rate = (
        Decimal(RATE_CARD["base"])
        + (fat - Decimal(RATE_CARD["reference_fat"]))
        * 10 * Decimal(RATE_CARD["fat"])
        + (snf - Decimal(RATE_CARD["reference_snf"]))
        * 10 * Decimal(RATE_CARD["snf"])
    )

    return rate.quantize(
        Decimal("0.01"),
        rounding=ROUND_HALF_UP
    )


# Calculate total payment
def calculate_payment(litres, fat, snf):
    litres = Decimal(str(litres))

    if litres <= 0:
        raise ValueError("Milk quantity must be positive")

    rate = calculate_milk_rate(fat, snf)
    payment = litres * rate

    return payment.quantize(
        Decimal("0.01"),
        rounding=ROUND_HALF_UP
    )


# Test the calculator
print("TASK 11: FINAL DESIGN RECOMMENDATION")
print("------------------------------------")

test_cases = [
    (10, 3.5, 8.5),
    (8.5, 4.2, 8.7),
    (5, 3.0, 8.0)
```

```
for litres, fat, snf in test_cases:
    rate = calculate_milk_rate(fat, snf)
    payment = calculate_payment(litres, fat, snf)

    print("\nLitres:", litres)
    print("Fat:", fat)
    print("SNF:", snf)
    print("Milk Rate: Rs.", rate)
    print("Payment: Rs.", payment)


# Demonstrate rounding
print("\nROUNDING RULE")
value = Decimal("10.125")

print("Original value:", value)
print(
    "ROUND_HALF_UP:",
    value.quantize(
        Decimal("0.01"),
        rounding=ROUND_HALF_UP
    )
)


# Demonstrate rate revision
print("\nRATE REVISION")
print("Old base rate: Rs.", RATE_CARD["base"])

RATE_CARD["base"] = "31.00"

with open(RATE_FILE, "w") as file:
    json.dump(RATE_CARD, file, indent=4)

RATE_CARD, file_size, load_time = load_rate_card()

print("New base rate: Rs.", RATE_CARD["base"])
print(
    "New rate for reference milk:",
    calculate_milk_rate(3.5, 8.5)
)
```

```
# JSON file experiment
print("\nJSON RATE CARD COMPARISON")
print("Rate card loaded from:", RATE_FILE)
print("JSON file size:", file_size, "bytes")
print("File loading time:", round(load_time, 2),
      "microseconds")


# Final recommendation
print("\nFINAL RECOMMENDATION")
print("1. Use Decimal for accurate monetary calculations.")
print("2. Use ROUND_HALF_UP for rounding to two decimals.")
print("3. Store rates in a JSON file for easier revisions.")
print("4. Validate inputs and test normal and invalid values.")
print("5. Keep rate changes approved and recorded.")
print("6. Use a database and audit logs at district scale.")


# Limitations
print("\nLIMITATIONS")
print("The program requires valid input and an approved rate card.")
print("It does not support multiple users or database storage.")
print("District deployment needs security, backups and testing.")

print("\nAll experiments completed.")
```